

Статистические методы и машинное обучение в Python: Полное руководство для анализа данных

Table of Contents

- [Обзор ключевых библиотек для анализа данных](#)
- [Классическое и геометрическое определение вероятности](#)
- [Классическое определение вероятности](#)
- [Задача: вероятность выпадения четного числа на кубике](#)
- [Геометрическая вероятность через симуляцию](#)
 - [Теоремы сложения и умножения вероятностей](#)
- [Теорема сложения: \$P\(A \cup B\) = P\(A\) + P\(B\)\$ для несовместных событий](#)
- [Теорема умножения: \$P\(A \cap B\) = P\(A\) \times P\(B\)\$ для независимых событий](#)
- [Для зависимых событий используется условная вероятность](#)
- [\$P\(A \cap B\) = P\(A|B\) \times P\(B\)\$](#)
 - [Условная вероятность и теорема Байеса](#)
- [Теорема Байеса: \$P\(A|B\) = P\(B|A\) \times P\(A\) / P\(B\)\$](#)
- [Пример: медицинский тест](#)
- [Формула полной вероятности](#)
- [Теорема Байеса](#)
- [Результат: 0.1667 \(всего 16.67% несмотря на точность теста 99%\)](#)
 - [Дискретные случайные величины](#)
- [Биномиальное распределение: количество успехов в n испытаниях](#)
- [Генерация выборки](#)
 - [Непрерывные случайные величины](#)
- [Нормальное \(Гауссово\) распределение](#)
- [Генерация выборки](#)
- [Экспоненциальное распределение \(время между событиями\)](#)
 - [Закон Больших Чисел и Центральная Предельная Теорема](#)
- [Закон Больших Чисел: демонстрация сходимости](#)
 - [Описательные статистики и работа с данными](#)
- [Описательные статистики через pandas](#)
- [Обнаружение выбросов \(метод IQR\)](#)
 - [Статистические оценки и их свойства](#)
- [Несмещенность оценок](#)
 - [Методы оценивания параметров](#)
- [Метод максимального правдоподобия через scipy](#)
 - [Доверительные интервалы](#)
- [Доверительный интервал для среднего](#)

- [t-распределение для малых выборок](#)
 - [Проверка статистических гипотез](#)
- [Одновыборочный t-тест \(\$H_0: \mu = \mu_0\$ \)](#)
- [Тест хи-квадрат на независимость](#)
- [Тест Шапиро-Уилка на нормальность](#)
 - [ANOVA \(дисперсионный анализ\)](#)
- [Генерируем данные для трех групп](#)
- [Односторонний ANOVA через scipy](#)
 - [Корреляция](#)
- [Генерируем коррелированные данные](#)
- [Корреляция Пирсона \(линейная зависимость\)](#)
- [Корреляция Спирмена \(ранговая\)](#)
 - [Линейная регрессия](#)
- [Генерируем данные](#)
- [STATSMODELS: подробная статистика](#)
- [SCIKIT-LEARN: предсказание](#)
 - [Логистическая регрессия](#)
- [Генерируем данные](#)
- [SCIKIT-LEARN](#)
 - [Временные ряды](#)
- [Создаем временной ряд](#)
- [Скользящее среднее](#)
- [Тест на стационарность](#)
- [ARIMA модель](#)
 - [Методы scikit-learn: статистический полный обзор](#)
 - [Дополнительные популярные библиотеки для анализа данных](#)
 - [Типичный workflow анализа данных](#)
 - [Ключевые выводы](#)

Как эксперт с 15-летним опытом решения бизнес-задач с помощью анализа данных и машинного обучения, представляю вам подробное руководство по основным библиотекам Python для статистического анализа. Это руководство охватывает все ключевые темы от теории вероятностей до временных рядов с практическими примерами кода, бизнес-применениями и детальным объяснением того, какие статистические методы реализованы в scikit-learn.

Обзор ключевых библиотек для анализа данных

Экосистема Python предлагает богатый набор библиотек для статистического анализа и машинного обучения. В 2025 году основными инструментами остаются **pandas**, **NumPy**, **SciPy**, **statsmodels** и **scikit-learn**, каждая из которых решает специфические задачи.

NumPy служит фундаментом для численных вычислений, предоставляя высокопроизводительные операции с многомерными массивами. **Pandas** специализируется на работе с табличными данными и предлагает мощные структуры данных **DataFrame** и **Series**. **SciPy** расширяет возможности NumPy, добавляя обширный набор статистических функций и распределений вероятностей.

Критическое различие между библиотеками заключается в их философии: **statsmodels** фокусируется на статистическом выводе и интерпретации (p-values, доверительные интервалы, статистические тесты), в то время как **scikit-learn** оптимизирован для предсказательного моделирования и автоматизации ML-процессов. Для глубокого понимания зависимостей используйте statsmodels, для production-моделей — scikit-learn.

Помимо основных библиотек, современные специалисты по данным также используют **Plotly** и **Seaborn** для визуализации, **XGBoost** и **LightGBM** для продвинутого машинного обучения, а **Prophet** для прогнозирования временных рядов.

Классическое и геометрическое определение вероятности

Вероятность — основа статистического анализа. Классическое определение применяется к равновероятным исходам, а геометрическое — к непрерывным пространствам событий. В Python для работы с вероятностями используется `numpy.random` для генерации случайных величин и `scipy.stats` для работы с распределениями.

```
import numpy as np
from scipy import stats

# Классическое определение вероятности<a></a>
# Задача: вероятность выпадения четного числа на кубике<a></a>
outcomes = np.array([1, 2, 3, 4, 5, 6])
favorable = np.array([2, 4, 6])
prob_classic = len(favorable) / len(outcomes)
print(f"Теоретическая вероятность: {prob_classic}") # 0.5

# Геометрическая вероятность через симуляцию<a></a>
np.random.seed(42)
rolls = np.random.choice(outcomes, size=10000)
prob_simulated = np.sum(np.isin(rolls, favorable)) / len(rolls)
print(f"Эмпирическая вероятность (10000 бросков): {prob_simulated:.4f}")
```

Бизнес-применение: Оценка рисков, расчет вероятности дефолта клиента, планирование запасов на основе вероятностных моделей спроса. Например, в страховании классическая вероятность используется для расчета премий, а симуляции Monte Carlo — для моделирования портфельных рисков.

Теоремы сложения и умножения вероятностей

Теоремы сложения (для несовместных событий) и умножения (для независимых событий) являются основополагающими для вероятностного моделирования. В Python эти операции реализуются через базовые арифметические операции с использованием `numpy`.

```
import numpy as np

# Теорема сложения:  $P(A \cup B) = P(A) + P(B)$  для несовместных событий<a></a>
p_a = 0.3 # вероятность события A
p_b = 0.2 # вероятность события B
p_union = p_a + p_b # для несовместных событий
print(f" $P(A \cup B) = {p\_union}$ ")

# Теорема умножения:  $P(A \cap B) = P(A) \times P(B)$  для независимых событий<a></a>
p_intersection = p_a * p_b
print(f" $P(A \cap B) = {p\_intersection}$ ")

# Для зависимых событий используется условная вероятность<a></a>
#  $P(A \cap B) = P(A|B) \times P(B)$ <a></a>
```

Бизнес-применение: Анализ конверсионных воронок, оценка комплексных рисков в проектном управлении, расчет надежности систем из независимых компонентов.

Условная вероятность и теорема Байеса

Условная вероятность и теорема Байеса критически важны для современного машинного обучения. Scipy.stats не предоставляет прямых функций для байесовских вычислений, но scikit-learn реализует наивный байесовский классификатор.

```
import numpy as np
from sklearn.naive_bayes import GaussianNB

# Теорема Байеса:  $P(A|B) = P(B|A) \times P(A) / P(B)$ 
# Пример: медицинский тест
p_disease = 0.01 # P(болезнь)
p_positive_given_disease = 0.99 # P(+|болезнь)
p_positive_given_no_disease = 0.05 # P(+|нет болезни)

# Формула полной вероятности
p_positive = (p_positive_given_disease * p_disease +
              p_positive_given_no_disease * (1 - p_disease))

# Теорема Байеса
p_disease_given_positive = (p_positive_given_disease * p_disease) / p_positive
print(f"P(болезнь|положительный тест) = {p_disease_given_positive:.4f}")
# Результат: 0.1667 (всего 16.67% несмотря на точность теста 99%)
```

Бизнес-применение: Спам-фильтрация email-рассылок, медицинская диагностика, кредитный скоринг, рекомендательные системы, A/B тестирование.

Методы sklearn: `sklearn.naive_bayes.GaussianNB` (для непрерывных признаков), `sklearn.naive_bayes.MultinomialNB` (для дискретных признаков), `sklearn.naive_bayes.BernoulliNB` (для бинарных признаков).

Дискретные случайные величины

Дискретные случайные величины моделируются через `scipy.stats`, который предоставляет более 80 распределений вероятностей. Основные методы: `pmf()` (функция вероятности), `cdf()` (функция распределения), `rvs()` (генерация выборки).

```
from scipy import stats
import numpy as np

# Биномиальное распределение: количество успехов в n испытаниях
n_trials = 10
p_success = 0.3
binomial_rv = stats.binom(n=n_trials, p=p_success)

print(f"Математическое ожидание: {binomial_rv.mean():.2f}") # n*p = 3.0
print(f"Дисперсия: {binomial_rv.var():.2f}") # n*p*(1-p) = 2.1
print(f"P(X=3): {binomial_rv.pmf(3):.4f}") # вероятность ровно 3 успехов
print(f"P(X≤3): {binomial_rv.cdf(3):.4f}") # вероятность не более 3 успехов

# Генерация выборки
samples = binomial_rv.rvs(size=1000, random_state=42)
print(f"Выборочное среднее: {np.mean(samples):.2f}")
```

Бизнес-применение: Биномиальное распределение — для моделирования конверсий, успешности звонков. Пуассон — для моделирования количества заказов в час, количества дефектов.

Непрерывные случайные величины

Непрерывные распределения моделируют величины с бесконечным числом возможных значений. Scipy.stats предоставляет `pdf()` (плотность вероятности), `cdf()`, `ppf()` (квантильная функция).

```
from scipy import stats
import numpy as np

# Нормальное (Гауссово) распределение<a></a>
mu, sigma = 100, 15 # среднее и стандартное отклонение
normal_rv = stats.norm(loc=mu, scale=sigma)

print(f"P(X ≤ 110): {normal_rv.cdf(110):.4f}")
print(f"P(85 ≤ X ≤ 115): {normal_rv.cdf(115) - normal_rv.cdf(85):.4f}")
print(f"95-й процентиль: {normal_rv.ppf(0.95):.2f}")

# Генерация выборки<a></a>
samples = normal_rv.rvs(size=1000, random_state=42)

# Экспоненциальное распределение (время между событиями)<a></a>
lambda_rate = 0.5
exp_rv = stats.expon(scale=1/lambda_rate)
print(f"\nСреднее время между событиями: {exp_rv.mean():.2f}")
```

Бизнес-применение: Нормальное распределение — для моделирования роста, веса, ошибок измерений. Экспоненциальное — для времени между покупками, времени обслуживания клиентов.

Закон Больших Чисел и Центральная Предельная Теорема

ЗБЧ гарантирует сходимость выборочного среднего к истинному при увеличении размера выборки. ЦПТ утверждает, что распределение выборочных средних приближается к нормальному независимо от исходного распределения.

```
import numpy as np
from scipy import stats

# Закон Больших Чисел: демонстрация сходимости<a></a>
np.random.seed(42)
true_prob = 0.5 # вероятность "орла" при броске монеты
sample_sizes = [10, 100, 1000, 10000, 100000]

print("Сходимость к истинной вероятности:")
for n in sample_sizes:
    tosses = np.random.binomial(1, true_prob, n)
    empirical_prob = np.mean(tosses)
    print(f"n={n:6d}: выборочное среднее = {empirical_prob:.5f}")
```

Бизнес-применение: ЗБЧ — основа выборочных исследований рынка, контроля качества. ЦПТ — обоснование для использования нормальных доверительных интервалов, страховой математики.

Описательные статистики и работа с данными

Описательная статистика дает первое представление о данных. Pandas метод `describe()` и `scipy.stats.describe` предоставляют комплексную информацию.

```
import pandas as pd
import numpy as np
from scipy import stats

# Описательные статистики через pandas<a></a>
```

```

print("Описательные статистики (pandas):")
print(df.describe())

# Обнаружение выбросов (метод IQR)
Q1 = df['продажи'].quantile(0.25)
Q3 = df['продажи'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df['продажи'] < lower_bound) | (df['продажи'] > upper_bound)]
print(f"Выбросы по методу IQR: {len(outliers)}")

```

Методы sklearn: `sklearn.impute.SimpleImputer`, `sklearn.impute.KNNImputer`, `sklearn.ensemble.IsolationForest` (выбросы), `sklearn.covariance.EllipticEnvelope`.

Статистические оценки и их свойства

Статистические оценки имеют важные свойства: несмещенность, состоятельность, эффективность. В NumPy параметр `ddof` контролирует смещенность оценки дисперсии.

```

import numpy as np

# Несмещенность оценок
population = np.random.normal(100, 15, 10000)
true_mean = 100
true_var = 15**2

sample_vars_biased = []
sample_vars_unbiased = []

for _ in range(1000):
    sample = np.random.choice(population, size=30, replace=False)
    sample_vars_biased.append(np.var(sample, ddof=0)) # смещенная оценка
    sample_vars_unbiased.append(np.var(sample, ddof=1)) # несмещенная

print(f"Истинная дисперсия: {true_var}")
print(f"Смещенная оценка (ddof=0): {np.mean(sample_vars_biased):.2f}")
print(f"Несмещенная оценка (ddof=1): {np.mean(sample_vars_unbiased):.2f}")

```

Методы оценивания параметров

ММ и MLE — основные методы оценки параметров распределений. Scipy.stats предоставляет метод `.fit()` для MLE-оценок.

```

from scipy import stats
import numpy as np

# Метод максимального правдоподобия через scipy
np.random.seed(42)
data_sample = np.random.normal(50, 10, 1000)

mu_mle, sigma_mle = stats.norm.fit(data_sample)
print("Метод максимального правдоподобия:")
print(f"  μ = {mu_mle:.4f}")
print(f"  σ = {sigma_mle:.4f}")

```

Доверительные интервалы

Доверительные интервалы количественно выражают неопределенность статистических оценок. Scipy.stats предоставляет методы для построения ДИ через t-распределение.

```
from scipy import stats
import numpy as np

# Доверительный интервал для среднего<a></a>
np.random.seed(42)
sample = np.random.normal(100, 15, 50)
n = len(sample)
mean = np.mean(sample)
std_err = stats.sem(sample)
confidence = 0.95

# t-распределение для малых выборок<a></a>
t_critical = stats.t.ppf((1 + confidence) / 2, df=n-1)
margin_of_error = t_critical * std_err
ci = (mean - margin_of_error, mean + margin_of_error)
print(f"95% ДИ для среднего: [{ci[0]:.2f}, {ci[1]:.2f}]" )
```

Бизнес-применение: Прогноз выручки с интервалом неопределенности для планирования бюджета, оценка конверсии с доверительными границами в A/B тестировании.

Проверка статистических гипотез

Статистические тесты — основа принятия решений на основе данных. Scipy.stats содержит полный набор параметрических и непараметрических тестов.

```
from scipy import stats
import numpy as np

# Одновыборочный t-тест ( $H_0: \mu = \mu_0$ )<a></a>
np.random.seed(42)
sample = np.random.normal(105, 15, 40)
t_stat, p_value = stats.ttest_1samp(sample, 100)
print("Одновыборочный t-тест:")
print(f"  t-статистика: {t_stat:.4f}, p-value: {p_value:.4f}")
print(f"  Вывод: {'Отвергаем H0' if p_value < 0.05 else 'Не отвергаем H0'}")

# Тест хи-квадрат на независимость<a></a>
contingency = np.array([[30, 10], [20, 40]])
chi2, p_chi, dof, expected = stats.chi2_contingency(contingency)
print(f"\nХи-квадрат:  $\chi^2={chi2:.2f}$ , p-value={p_chi:.4f}")

# Тест Шапиро-Уилка на нормальность<a></a>
normal_data = np.random.normal(0, 1, 100)
w_stat, p_shapiro = stats.shapiro(normal_data)
print(f"Шапиро-Уилка: p-value={p_shapiro:.4f}")
```

Бизнес-применение: A/B тестирование веб-сайтов, оценка эффекта обучения персонала, контроль качества продукции.

Важно: Scikit-learn НЕ содержит статистических тестов — используйте scipy.stats.

ANOVA (дисперсионный анализ)

ANOVA используется для сравнения средних трех и более групп. Scipy.stats предоставляет `f_oneway()`, а statsmodels — детальные таблицы.

```
from scipy import stats
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols

# Генерируем данные для трех групп<a></a>
np.random.seed(42)
group_A = np.random.normal(100, 15, 30)
group_B = np.random.normal(110, 15, 30)
group_C = np.random.normal(105, 15, 30)

# Односторонний ANOVA через scipy<a></a>
f_stat, p_anova = stats.f_oneway(group_A, group_B, group_C)
print("Односторонний ANOVA:")
print(f" F-статистика: {f_stat:.4f}, p-value: {p_anova:.4f}")
```

Бизнес-применение: Сравнение эффективности нескольких версий продукта, анализ продаж по регионам.

Корреляция

Корреляционный анализ выявляет линейные и монотонные зависимости между переменными.

```
from scipy import stats
import pandas as pd
import numpy as np

# Генерируем коррелированные данные<a></a>
np.random.seed(42)
x = np.random.normal(0, 1, 100)
y = 2 * x + np.random.normal(0, 0.5, 100)

# Корреляция Пирсона (линейная зависимость)<a></a>
corr_xy, p_xy = stats.pearsonr(x, y)
print("Корреляция Пирсона:")
print(f" r(x,y) = {corr_xy:.4f}, p-value = {p_xy:.4f}")

# Корреляция Спирмена (ранговая)<a></a>
corr_spearman, p_spearman = stats.spearmanr(x, y)
print(f"Спирмен: ρ(x,y) = {corr_spearman:.4f}, p-value = {p_spearman:.4f}")
```

Бизнес-применение: Анализ факторов лояльности клиентов, feature selection для ML-моделей.

Линейная регрессия

Линейная регрессия — один из важнейших методов анализа данных. Statsmodels дает полную статистическую информацию, sklearn фокусируется на предсказаниях.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.api as sm

# Генерируем данные<a></a>
```

```

np.random.seed(42)
n = 100
X1 = np.random.uniform(20, 80, n)
X2 = np.random.uniform(1, 10, n)
y = 50 + 2*X1 - 3*X2 + np.random.normal(0, 10, n)

df_reg = pd.DataFrame({'X1': X1, 'X2': X2, 'y': y})

# STATSMODELS: подробная статистика<a></a>
X_sm = df_reg[['X1', 'X2']]
X_sm = sm.add_constant(X_sm)
y_sm = df_reg['y']
model_sm = sm.OLS(y_sm, X_sm).fit()
print(model_sm.summary())

# SCIKIT-LEARN: предсказание<a></a>
X_sk = df_reg[['X1', 'X2']].values
y_sk = df_reg['y'].values
X_train, X_test, y_train, y_test = train_test_split(X_sk, y_sk, test_size=0.2, random_state=42)

model_sk = LinearRegression()
model_sk.fit(X_train, y_train)
y_pred = model_sk.predict(X_test)

print(f"\nR²: {r2_score(y_test, y_pred):.4f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")

```

Бизнес-применение: Прогнозирование продаж, ценообразование, оценка влияния факторов.

Дополнительные методы sklearn: Ridge, Lasso, ElasticNet, BayesianRidge, SGDRegressor.

Логистическая регрессия

Логистическая регрессия используется для бинарной и многоклассовой классификации.

```

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
import statsmodels.api as sm

# Генерируем данные<a></a>
np.random.seed(42)
n = 200
X1 = np.random.uniform(20, 150, n)
X2 = np.random.uniform(300, 850, n)
logit = -10 + 0.05*X1 + 0.01*X2
prob = 1 / (1 + np.exp(-logit))
y = (prob > 0.5).astype(int)

df = pd.DataFrame({'X1': X1, 'X2': X2, 'y': y})

# SCIKIT-LEARN<a></a>
X = df[['X1', 'X2']].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

logit_model = LogisticRegression(random_state=42, max_iter=1000)
logit_model.fit(X_train, y_train)

y_pred = logit_model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, logit_model.predict_proba(X_test)[:, 1]):.4f}")

```

Бизнес-применение: Кредитный скоринг, прогнозирование оттока клиентов, fraud detection.

Временные ряды

Анализ временных рядов требует специализированных методов. Statsmodels.tsa предоставляет ARIMA, SARIMA, экспоненциальное сглаживание.

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
from statsmodels.tsa.arima.model import ARIMA

# Создаем временной ряд<a></a>
dates = pd.date_range('2020-01-01', periods=365, freq='D')
np.random.seed(42)
trend = np.linspace(100, 150, 365)
seasonal = 10 * np.sin(2 * np.pi * np.arange(365) / 365 * 4)
noise = np.random.normal(0, 5, 365)
ts_values = trend + seasonal + noise

df_ts = pd.DataFrame({'значение': ts_values}, index=dates)

# Скользящее среднее<a></a>
df_ts['MA_7'] = df_ts['значение'].rolling(window=7).mean()

# Тест на стационарность<a></a>
adf_result = adfuller(df_ts['значение'])
print(f"ADF статистика: {adf_result[0]:.4f}, p-value: {adf_result[1]:.4f}")

# ARIMA модель<a></a>
arima_model = ARIMA(df_ts['значение'], order=(1, 0, 1))
arima_fit = arima_model.fit()
forecast = arima_fit.forecast(steps=7)
print(f"Прогноз на 7 дней:\n{forecast}")
```

Бизнес-применение: Прогнозирование спроса, финансовые прогнозы, прогноз нагрузки на серверы.

Важно: Scikit-learn НЕ содержит методов для временных рядов — используйте statsmodels.tsa или Prophet.

Методы scikit-learn: статистический полный обзор

Scikit-learn фокусируется на машинном обучении и содержит следующие статистически-ориентированные компоненты:

Наивный Байес (теорема Байеса)

- `sklearn.naive_bayes.GaussianNB` — для непрерывных признаков
- `sklearn.naive_bayes.MultinomialNB` — для дискретных признаков
- `sklearn.naive_bayes.BernoulliNB` — для бинарных признаков

Линейная регрессия и вариации

- `sklearn.linear_model.LinearRegression` — классический МНК
- `sklearn.linear_model.Ridge` — L2-регуляризация
- `sklearn.linear_model.Lasso` — L1-регуляризация
- `sklearn.linear_model.ElasticNet` — комбинированная регуляризация
- `sklearn.linear_model.BayesianRidge` — байесовская регрессия

Логистическая регрессия

- `sklearn.linear_model.LogisticRegression` — с поддержкой регуляризации

Работа с пропущенными данными

- `sklearn.impute.SimpleImputer` — mean, median, most_frequent
- `sklearn.impute.KNNImputer` — на основе соседей
- `sklearn.impute.IterativeImputer` — MICE

Обнаружение выбросов

- `sklearn.ensemble.IsolationForest`
- `sklearn.neighbors.LocalOutlierFactor`
- `sklearn.covariance.EllipticEnvelope`
- `sklearn.svm.OneClassSVM`

Стандартизация и нормализация

- `sklearn.preprocessing.StandardScaler` — z-score
- `sklearn.preprocessing.RobustScaler` — устойчивая к выбросам
- `sklearn.preprocessing.PowerTransformer` — Box-Cox
- `sklearn.preprocessing.QuantileTransformer`

Валидация и разделение данных

- `sklearn.model_selection.train_test_split`
- `sklearn.model_selection.cross_val_score`
- `sklearn.model_selection.StratifiedKFold`
- `sklearn.model_selection.TimeSeriesSplit`

Метрики качества

Регрессия: MSE, MAE, R^2 , MAPE

Классификация: accuracy, precision, recall, F1, ROC-AUC

Feature selection

- `sklearn.feature_selection.f_regression` — F-статистика
- `sklearn.feature_selection.chi2` — тест хи-квадрат
- `sklearn.feature_selection.SelectKBest`

Оценка ковариаций

- `sklearn.covariance.EmpiricalCovariance`
- `sklearn.covariance.LedoitWolf`

Что НЕ содержит sklearn:

- Статистические тесты → используйте `scipy.stats`
- P-values и доверительные интервалы → используйте `statsmodels`
- Временные ряды (ARIMA) → используйте `statsmodels.tsa`

- Подробную диагностику моделей → используйте statsmodels

Дополнительные популярные библиотеки для анализа данных

Визуализация

- **Plotly** — интерактивные графики для дашбордов
- **Seaborn** — статистическая визуализация
- **Matplotlib** — базовая визуализация

Продвинутое ML

- **XGBoost, LightGBM, CatBoost** — градиентный бустинг
- **Prophet** — прогнозирование временных рядов

Статистика

- **Pingouin** — альтернатива scipy с простым API

Дисбаланс классов

- **Imbalanced-learn** — SMOTE и другие методы

Большие данные

- **Polars** — современная альтернатива pandas
- **Dask** — параллельные вычисления

Типичный workflow анализа данных

1. Загрузка и первичный осмотр → pandas
2. EDA и описательная статистика → pandas + seaborn
3. Обработка пропусков и выбросов → sklearn
4. Статистические тесты → scipy.stats
5. Feature engineering → sklearn.preprocessing
6. Построение модели:
 - Интерпретация параметров → statsmodels
 - Предсказание → sklearn
7. Валидация модели → sklearn.model_selection
8. Визуализация результатов → matplotlib/seaborn/plotly

Ключевые выводы

Экосистема Python для анализа данных предоставляет взаимодополняющие инструменты. NumPy и pandas обеспечивают базовую работу с данными, SciPy предоставляет статистические тесты и распределения, statsmodels специализируется на статистическом выводе, а scikit-learn оптимизирован для предсказательного моделирования.

Ключевое различие: statsmodels отвечает на вопрос "почему?" (статистическая значимость, p-values), sklearn — на "что?" и "насколько хорошо предсказывает?" (кросс-валидация, производительность).

В практической работе эти библиотеки комбинируются: используйте statsmodels для исследовательского анализа, sklearn — для production-моделей. Понимание сильных сторон каждого инструмента позволяет выбрать оптимальное решение для конкретной бизнес-задачи.